Modul 1 — DDL (Data Definition Language)

Mata pelajaran: Basis Data — RPL **Alat:** MySQL/MariaDB (XAMPP), phpMyAdmin atau terminal **Studi kasus:** Sistem data sekolah (`db_sekolah`)

---

1. Tujuan Pembelajaran

Setelah mempelajari modul ini, kamu mampu:

1. Menjelaskan pengertian dan fungsi DDL.
2. Membuat database dan tabel lengkap dengan tipe data dan constraint.
3. Membuat relasi antartabel menggunakan primary key dan foreign key.
4. Mengubah struktur tabel dengan `ALTER TABLE`.
5. Membedakan `DELETE`, `TRUNCATE`, dan `DROP`.

---

2. Konsep Dasar

DDL (Data Definition Language) adalah kelompok perintah SQL untuk **mendefinisikan struktur** database: membuat, mengubah, dan menghapus database atau tabel. DDL tidak berfokus pada isi data, melainkan pada "wadah" penyimpan data.

Perintah	Fungsi singkat
`CREATE`	Membuat database/tabel
`ALTER`	Mengubah struktur tabel
`DROP`	Menghapus database/tabel beserta isinya
`TRUNCATE`	Mengosongkan seluruh isi tabel
`RENAME`	Mengganti nama tabel

Analogi: DDL seperti membangun rak arsip. Kamu menentukan jumlah rak, label, dan sekat, sebelum memasukkan berkas apa pun.

Istilah penting

Istilah	Arti
Database	Wadah yang menampung banyak tabel
Tabel	Kumpulan data sejenis (baris dan kolom)
Kolom	Atribut data, misalnya `nama`
Baris	Satu catatan data, misalnya satu siswa
Primary key	Identitas unik setiap baris
Foreign key	Kolom penghubung ke tabel lain

3. Studi Kasus

Sekolah ingin menyimpan data **jurusan, siswa, mata pelajaran, dan nilai**. Rancangan relasinya:

```
jurusan (1) ———< siswa (N)
siswa   (1) ———< nilai (N) >——— (1) mapel
```

- Satu jurusan memiliki banyak siswa.
- Satu siswa memiliki banyak nilai.
- Satu mapel memiliki banyak nilai.

Tugas kita: membangun struktur database tersebut dengan DDL.

---

4. Persiapan

1. Buka XAMPP Control Panel, klik **Start** pada MySQL.
2. Buka `http://localhost/phpmyadmin`, pilih tab **SQL**. Atau lewat terminal:

   ```
   cd C:\xampp\mysql\bin
   mysql -u root -p
   ```

   `cd` berpindah ke folder program MySQL, sedangkan `mysql -u root -p` membuka klien MySQL sebagai pengguna `root`. Opsi `-p` akan meminta kata sandi.
3. Jalankan query satu per satu sesuai urutan modul. Akhiri setiap perintah SQL dengan titik koma (`;`).

---

5. Materi dan Praktik

5.1 `CREATE DATABASE` — membuat database

```
CREATE DATABASE IF NOT EXISTS db_sekolah; -- buat database jika belum ada
USE db_sekolah;                           -- pilih database untuk perintah berikutnya
```

Arti perintah:

- `CREATE DATABASE` meminta MySQL membuat database baru bernama `db_sekolah`.
- `IF NOT EXISTS` membuat perintah tetap aman dijalankan jika database tersebut sudah ada; perintah ini tidak menghapus atau mengosongkan database yang sudah ada.
- `USE` menetapkan `db_sekolah` sebagai database aktif. Perintah berikutnya yang tidak menyebut nama database akan dijalankan di database ini.

Memeriksa hasil:

```
SHOW DATABASES;      -- tampilkan daftar database yang dapat diakses
SELECT DATABASE();   -- tampilkan nama database yang sedang dipilih
```

`SHOW DATABASES` dan `SELECT DATABASE()` hanya membaca informasi, tidak mengubah struktur maupun data.

5.2 `CREATE TABLE` — membuat tabel

a. Tabel `jurusan` (tabel induk)

```
CREATE TABLE IF NOT EXISTS jurusan (
    id_jurusan   INT UNSIGNED AUTO_INCREMENT PRIMARY KEY, -- ID unik, dibuat otomatis
    nama_jurusan VARCHAR(100) NOT NULL UNIQUE             -- nama wajib diisi dan tidak boleh kembar
);
```

`CREATE TABLE` membuat wadah bernama `jurusan`. `IF NOT EXISTS` mencegah error jika tabel dengan nama tersebut sudah ada, tetapi tidak memperbarui struktur tabel lama. Tabel ini tidak bergantung pada tabel lain, sehingga dapat dibuat paling awal.

b. Tabel `siswa` (memiliki foreign key)

```
CREATE TABLE IF NOT EXISTS siswa (
    id_siswa      INT UNSIGNED AUTO_INCREMENT PRIMARY KEY, -- ID unik siswa yang bertambah otomatis
    nama          VARCHAR(100) NOT NULL,                   -- nama wajib diisi, maksimal 100 karakter
    email         VARCHAR(150) UNIQUE,                     -- email opsional, tetapi tidak boleh kembar
    jenis_kelamin ENUM('L', 'P') NOT NULL,                 -- hanya menerima L atau P dan wajib diisi
    tanggal_lahir DATE,                                     -- tanggal opsional dengan format YYYY-MM-DD
    id_jurusan    INT UNSIGNED,                            -- boleh NULL jika belum memiliki jurusan
    created_at    TIMESTAMP DEFAULT CURRENT_TIMESTAMP,     -- otomatis berisi waktu pembuatan baris

    CONSTRAINT fk_siswa_jurusan
        FOREIGN KEY (id_jurusan)              -- kolom penghubung pada tabel siswa
        REFERENCES jurusan(id_jurusan)        -- harus cocok dengan ID pada tabel jurusan
        ON UPDATE CASCADE                     -- perubahan ID jurusan ikut diterapkan di siswa
        ON DELETE SET NULL                    -- penghapusan jurusan mengosongkan id_jurusan siswa
);
```

Nama constraint `fk_siswa_jurusan` membantu mengenali aturan foreign key. Karena kebijakannya `ON DELETE SET NULL`, kolom `id_jurusan` memang harus mengizinkan `NULL`; baris siswa tidak ikut terhapus saat jurusannya dihapus.

c. Tabel `mapel`

```
CREATE TABLE IF NOT EXISTS mapel (
    id_mapel   INT UNSIGNED AUTO_INCREMENT PRIMARY KEY, -- ID unik mata pelajaran
    nama_mapel VARCHAR(100) NOT NULL UNIQUE             -- nama wajib diisi dan tidak boleh kembar
);
```

Tabel `mapel` juga berdiri sendiri, sehingga dibuat sebelum tabel `nilai` yang akan merujuk kepadanya.

d. Tabel `nilai` (dua foreign key)

```
CREATE TABLE IF NOT EXISTS nilai (
    id_nilai    INT UNSIGNED AUTO_INCREMENT PRIMARY KEY, -- ID unik setiap catatan nilai
    id_siswa    INT UNSIGNED NOT NULL,                   -- siswa pemilik nilai, wajib diisi
    id_mapel    INT UNSIGNED NOT NULL,                   -- mata pelajaran yang dinilai, wajib diisi
    nilai_angka DECIMAL(5,2) NOT NULL,                   -- angka dengan dua digit di belakang koma
    semester    INT UNSIGNED NOT NULL,               -- nomor semester berupa bilangan nonnegatif

    CONSTRAINT fk_nilai_siswa
        FOREIGN KEY (id_siswa) REFERENCES siswa(id_siswa) -- pastikan siswa yang dirujuk benar-benar ada
        ON UPDATE CASCADE ON DELETE CASCADE,              -- ID siswa ikut berubah; nilainya ikut terhapus

    CONSTRAINT fk_nilai_mapel
        FOREIGN KEY (id_mapel) REFERENCES mapel(id_mapel) -- pastikan mapel yang dirujuk benar-benar ada
        ON UPDATE CASCADE ON DELETE CASCADE               -- ID mapel ikut berubah; nilainya ikut terhapus
);
```

`DECIMAL(5,2)` dapat menyimpan sampai lima digit total dengan dua digit desimal, misalnya `100.00`. Jika siswa atau mapel dihapus, catatan nilai yang merujuk kepadanya ikut terhapus karena aturan `ON DELETE CASCADE`.

> **Urutan pembuatan penting:** tabel induk (`jurusan`, `mapel`) harus ada sebelum tabel yang merujuknya (`siswa`, `nilai`).

5.3 Arti tipe data dan constraint

Sintaks	Penjelasan
`INT UNSIGNED`	Bilangan bulat nonnegatif
`AUTO_INCREMENT`	Nilai bertambah otomatis saat baris baru dibuat
`PRIMARY KEY`	Unik dan tidak boleh `NULL`
`VARCHAR(100)`	Teks maksimal 100 karakter
`NOT NULL`	Wajib diisi
`UNIQUE`	Tidak boleh ada nilai kembar (nilai `NULL` boleh lebih dari satu)
`ENUM('L','P')`	Hanya boleh bernilai `L` atau `P`
`DATE`	Tanggal, format `YYYY-MM-DD`
`DECIMAL(5,2)`	Maksimal 5 digit, 2 di antaranya desimal (contoh: 100.00)
`TIMESTAMP DEFAULT CURRENT_TIMESTAMP`	Terisi waktu saat baris dibuat
`FOREIGN KEY`	Menjaga hubungan dengan tabel lain
`ON DELETE CASCADE`	Data anak ikut terhapus saat induk dihapus
`ON DELETE SET NULL`	Kolom penghubung dikosongkan saat induk dihapus

5.4 Memeriksa struktur: `SHOW` dan `DESCRIBE`

```
SHOW TABLES;              -- tampilkan tabel pada database aktif
DESCRIBE siswa;           -- ringkas nama kolom, tipe, key, dan izin NULL
SHOW CREATE TABLE siswa;  -- tampilkan definisi SQL lengkap beserta constraint
```

`DESCRIBE` membantu memeriksa apakah definisi kolom sesuai rancangan; `SHOW CREATE TABLE` berguna untuk memeriksa aturan lengkap seperti foreign key. Hasil yang diharapkan dari `SHOW TABLES` adalah 4 tabel: `jurusan`, `mapel`, `nilai`, dan `siswa`.

5.5 `ALTER TABLE` — mengubah struktur

Kasus: sekolah baru menyadari perlu menyimpan nomor telepon siswa.

```
-- 1. Menambah kolom
ALTER TABLE siswa
ADD COLUMN no_hp VARCHAR(20); -- nomor telepon opsional dengan panjang hingga 20 karakter

-- 2. Mengubah tipe/ukuran kolom
ALTER TABLE siswa
MODIFY COLUMN nama VARCHAR(150) NOT NULL; -- ubah batas nama menjadi 150 karakter; tetap wajib diisi

-- 3. Mengganti nama kolom
ALTER TABLE siswa
RENAME COLUMN no_hp TO nomor_telepon; -- nama kolom berubah, isi datanya tetap

-- 4. Menghapus kolom
ALTER TABLE siswa
DROP COLUMN nomor_telepon; -- hapus kolom beserta seluruh nilai di dalamnya
```

Setiap `ALTER TABLE` mengubah definisi tabel. Jalankan satu perubahan dalam satu waktu, lalu periksa hasilnya dengan `DESCRIBE siswa;`.

Arti bagian perintah:

Perintah	Fungsi
`ADD COLUMN`	Menambah kolom; baris lama berisi `NULL`
`MODIFY COLUMN`	Mengubah definisi kolom
`RENAME COLUMN`	Mengganti nama kolom
`DROP COLUMN`	Menghapus kolom beserta datanya

> **Catatan versi:** `RENAME COLUMN` tersedia di MySQL 8.0+ dan MariaDB 10.5.2+. Jika error, gunakan:
>
> ```
> ALTER TABLE siswa CHANGE COLUMN no_hp nomor_telepon VARCHAR(20);
> ```
>
> Cek versi dengan `SELECT VERSION();`

5.6 `RENAME TABLE` — mengganti nama tabel

Gunakan tabel latihan terpisah agar tabel utama tidak terganggu:

```
CREATE TABLE IF NOT EXISTS latihan_tmp (
    id INT PRIMARY KEY,          -- nomor identitas latihan
    keterangan VARCHAR(50)       -- keterangan singkat
);

RENAME TABLE latihan_tmp TO latihan_baru; -- ubah nama tabel tanpa mengganti nama kolom
```

`RENAME TABLE nama_lama TO nama_baru` hanya mengganti nama tabel; definisi kolom dan datanya tetap ada.

5.7 `DROP`, `TRUNCATE`, dan `DELETE` — jangan tertukar

Perintah	Yang terjadi	Struktur tabel	Bisa pakai `WHERE`
`DELETE` (DML)	Menghapus baris tertentu	Tetap	Ya
`TRUNCATE`	Mengosongkan semua baris	Tetap	Tidak
`DROP TABLE`	Menghapus tabel total	Hilang	Tidak
`DROP DATABASE`	Menghapus database total	Hilang	Tidak

Praktik aman pada tabel latihan:

```
TRUNCATE TABLE latihan_baru;          -- hapus semua baris, tetapi pertahankan struktur tabel
DROP TABLE IF EXISTS latihan_baru;    -- hapus tabel beserta strukturnya jika tabel ada
```

> **Peringatan:** `DROP` dan `TRUNCATE` merupakan perintah DDL yang umumnya tidak bisa dibatalkan dengan `ROLLBACK`. Tabel yang direferensikan foreign key juga dapat menolak `TRUNCATE` / `DROP`. Jangan jalankan pada tabel `siswa`, `nilai`, dan lainnya setelah terisi data, kecuali memang ingin mengulang dari awal.

---

6. Rangkuman

Query	Tujuan
`CREATE DATABASE`	Membuat database
`USE`	Memilih database aktif
`CREATE TABLE`	Membuat tabel
`SHOW` / `DESCRIBE`	Memeriksa struktur
`ALTER TABLE`	Menambah, mengubah, menghapus kolom
`RENAME TABLE`	Mengganti nama tabel
`TRUNCATE TABLE`	Mengosongkan isi tabel
`DROP TABLE` / `DROP DATABASE`	Menghapus tabel / database

---

7. Latihan

1. Buat database `db_perpustakaan`.
2. Buat tabel `kategori` (`id_kategori` PK auto increment, `nama_kategori` wajib dan unik).
3. Buat tabel `buku` (`id_buku`, `judul` wajib, `pengarang`, `tahun_terbit` bertipe `YEAR`, `id_kategori` sebagai FK ke `kategori`).
4. Tambahkan kolom `penerbit VARCHAR(100)` ke tabel `buku`.
5. Ubah panjang `judul` menjadi 200 karakter.
6. Tampilkan struktur tabel `buku`.
7. Jelaskan perbedaan `DELETE`, `TRUNCATE`, dan `DROP` dengan kata-katamu sendiri.

Pertanyaan diskusi: mengapa tabel `kategori` harus dibuat sebelum tabel `buku`?

---

8. Kesalahan Umum

Masalah	Penyebab	Solusi
`Table already exists`	Tabel sudah dibuat	Gunakan `IF NOT EXISTS`
`Can't create table ... errno 150`	Tipe kolom FK berbeda dengan PK, atau tabel induk belum ada	Samakan tipe (`INT UNSIGNED`) dan urutan pembuatan
`No database selected`	Lupa `USE`	Jalankan `USE db_sekolah;`
`Cannot truncate a table referenced in a foreign key`	Ada FK yang merujuk	Hapus data anak dulu, atau gunakan tabel latihan